

ENGR 102 Summer Bridge**Day 16 Instructor Key**

Indexed Mutation and Independent Exam 2 Rehearsal

Monday, July 27, 2026 • 50 points

Name		UIN		Section	
------	--	-----	--	---------	--

Boolean-operator convention. Python operators appear as [and](#), [or](#), and [not](#). Their lowercase spelling is required in code.

Daily target**increasing independence**

Design, mutate, summarize, and test a complete list-processing program from requirements alone.

Allowed tools	All Exam 2 tools: loops, conditions, list reads, append, len, and indexed assignment
Support supplied	Only the requirements, constraints, and test obligations are supplied.
You must decide	The entire algorithm, variable inventory, loop form, mutation order, summaries, and tests.
Scaffold level	Independent construction; no planning questions and no algorithm steps.

Scoring and completion standard**50 points**

Evidence	Points	Secure work shows
Integrated trace	8	intermediate state, condition results, and exact output
Diagnosis and repair	6	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	36	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace**8 points**

Produce one complete mutation ledger and exact final output.

```
values = [-2, 7, 14, 3]
changed = 0

for index in range(len(values)):
    original = values[index]
    if original < 0:
        values[index] = 0
    elif original % 2 == 1:
        values[index] = original * 2
    else:
        continue
    changed = changed + 1

print(values, changed)
```

Your task

1. For every index, record original value, selected branch, final value, list state, and changed; then give exact output.

Answer and explanation

1. Index 0: original -2, negative branch, final 0, list [0, 7, 14, 3], changed 1. Index 1: original 7, odd branch, final 14, list [0, 14, 14, 3], changed 2. Index 2: original 14, else/continue, list unchanged, changed remains 2. Index 3: original 3, odd branch, final 6, list [0, 14, 14, 6], changed 3. Exact output: [0, 14, 14, 6] 3.

Task 2. Diagnose and repair**6 points**

The intent is to replace every negative value, including the last item, and count each replacement.

```
values = [4, -2, 8, -5]
changed = 0

for index in range(len(values) - 1):
    if values[index] < 0:
        values[index] = 0
        changed = changed + 1

print(values, changed)
```

Your task

1. Give actual and intended output, identify the boundary defect, repair it, and name a test that would fail if the last index were skipped again.

Answer and explanation

1. Actual output is [4, 0, 8, -5] 1. Intended output is [4, 0, 8, 0] 2. range(len(values) - 1) stops before the last valid index; repair with range(len(values)). Protective test [-1] must become [0] with changed 1.

Task 3. Construct a complete program**36 points across Pages 4-5****Program specification**

- Use temperatures = [18, -4, 31, 22, 50].
- Mutate every value below 0 to 0 and every value above 40 to 40.
- Count how many positions change.
- Using the final mutated values, compute total, average, and the count from 20 through 40 inclusive.
- Print the final list, changed count, total, average, and in_range count in that order on separate lines.
- Do not use sum, min, max, comprehensions, slicing, or helper functions. Provide two tests with expected results, including a no-change case and a first/last-change case.

Answer and explanation

```
temperatures = [18, -4, 31, 22, 50]
changed = 0
total = 0
in_range = 0

for index in range(len(temperatures)):
    if temperatures[index] < 0:
        temperatures[index] = 0
        changed = changed + 1
    elif temperatures[index] > 40:
        temperatures[index] = 40
        changed = changed + 1

    total = total + temperatures[index]
    if 20 <= temperatures[index] <= 40:
        in_range = in_range + 1

average = total / len(temperatures)
print(temperatures)
print(changed)
print(total)
print(average)
print(in_range)
```

Each position is mutated before it contributes to total or in_range, so every summary describes final state rather than original state.

An index loop is required because assignment must write back to a list position. The average is safe because the supplied list is nonempty.

Task 3 continued. Test and defend the program**included in 36 points****Expected tests and results**

1. Supplied list -> [18, 0, 31, 22, 40], changed 2, total 111, average 22.2, in_range 3.
2. No-change [20, 30] -> [20, 30], changed 0, total 50, average 25.0, in_range 2.
3. First/last change [-1, 20, 45] -> [0, 20, 40], changed 2, total 60, average 20.0, in_range 2.

Scoring guidance

6 correct traversal and writable indexes; 8 complete boundary mutation; 5 changed count; 6 final-state total/average; 4 inclusive range count; 3 exact output; 4 tests and justification.

Accept alternative correct code when it obeys the stated tool boundary, handles the required cases, and produces the required output. All blue text is answer or scoring guidance.

VIP Lab Alignment

Bridge Day 16 • Contract 2d4127aac856

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

Why this page is here

VIP Lab Day(s): 6

Activities: 18.25, 18.26, 18.27

Rehearse indexed traversal, filtering, compound Boolean evidence, and the Day 6 Independent Program Studio inside the announced loop/list Exam 2 boundary; do not introduce function structure today.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.25 18-25-work / 18-25-transfer / 18-25-cold	Reverse Values	Create a new list without mutating the input.
18.26 18-26-work / 18-26-transfer / 18-26-cold	Values At Or Above Cut-off	Filter without reordering.
18.27 18-27-work / 18-27-transfer / 18-27-cold	Feasible Workstations	Use one index across parallel workstation lists.

Readiness check before you code

- For Activity 18.25, write the exact index sequence used to visit every value in reverse order without changing the original list.

Answer and explanation: Start at `len(values) - 1`, stop after index 0, and step by `-1`. The original list is read only; each visited value is appended to a separate result list.

- For Activities 18.26 and 18.27, state the keep condition and name one boundary case that must be tested before Exam 2.

Answer and explanation: 18.26 keeps values at or above the inclusive cutoff. 18.27 keeps a workstation name only when the same index satisfies both inclusive constraints. Strong tests include equality at each boundary and a case that satisfies only one constraint.

Scope boundary. Day 16 is Exam 2 preparation. Functions are deliberately deferred until the Day 18 post-exam handoff.
--

Notebook and submission procedure

- Use the sample and transfer cells for formative practice; attempt before opening a reveal.
- Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only the Studio cells listed above are scored for independent mastery in the matching Lab Day notebook.
- Save or download the completed notebook as one `.ipynb` file.
- Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.